Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3
з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
“Розробка інтерфейса користувача”

Виконав:

Студент групи ІМ-55
Кузьменко Артур Олегович
Номер в списку - 11

Перевірив:

Київ 2026

Завдання:
1. Створити у середовищі розробки проект з ім’ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіант завдання (Лабораторна робота №3):

Оскільки для попередньої лабораторної роботи номер за журналом був Ж = 11, для вибору варіанту поточної роботи використовується значення Ж = Ж_лаб2 + 1, отже розрахунковий номер варіанту **Ж = 12**. Згідно з методичними вказівками, вимоги щодо фігур повторюються із новим значенням Ж.

1. Індивідуальні вимоги до малювання фігур (Ж = 12):

- **Сховище об'єктів:** Динамічний масив вказівників Shape **pcshape обсягом 112 об'єктів (оскільки 12 mod 3 = 0, а розмір N = 12 + 100 = 112).
- **"Гумовий" слід при вводі об'єктів:** Суцільна лінія чорного кольору (оскільки 12 mod 4 = 0).
- **Прямокутник:**
  - Ввід по двох протилежних кутах (оскільки 12 mod 2 = 0).
  - Відображення: чорний контур з кольоровим заповненням (оскільки 12 mod 5 = 2).
  - Колір заповнення: жовтий (оскільки 12 mod 6 = 0).
- **Еліпс:**
  - Ввід від центру до одного з кутів охоплюючого прямокутника (оскільки 12 mod 2 = 0).
  - Відображення: чорний контур еліпсу без заповнення (оскільки 12 mod 5 = 2).

- **Позначка поточного типу об'єкту:** Здійснюється в меню (метод `OnInitMenuPopup`), оскільки 12 mod 2 = 0.

2. Вимоги до об'єктно-орієнтованої архітектури вікна:

- Усі методи-обробники повідомлень (зокрема, і метод `OnNotify`) повинні бути функціями-членами спеціально розробленого класу вікна (наприклад, `EditorWindow`).
- У звіті обов'язково повинна бути наведена схема успадкування класів — діаграма класів.

3. Вимоги до інтерфейсу користувача (Toolbar та меню):

- У головному вікні повинно бути вікно Toolbar з кнопками, що відповідають типам об'єктів (Крапка, Лінія, Прямокутник, Еліпс) та дублюють підпункти меню "Об'єкти".
- Кнопки на панелі Toolbar повинні бути не з текстовим підписом, а з іконкою, за якою можна здогадатися про інструмент.
- Кнопки панелі Toolbar обов'язково мають бути забезпечені спливаючими підказками (tooltips).
- Меню "Об'єкти" повинно розташовуватися праворуч меню "Файл" та ліворуч меню "Довідка".
- Підпункти меню "Об'єкти" повинні містити назви геометричних форм українською мовою.

Вихідний текст файлів:

main.cpp:

```
#include <windows.h>

#include "editor.h"



int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow) {

    //створює екземпляр класу-редактора

    EditorWindow editor(hInstance);



    //створює вікно

    if (!editor.Create(L"Графічний редактор",
WS_OVERLAPPEDWINDOW)) {

        return 0;

    }



    //показує вікно

    ShowWindow(editor.Window(), nCmdShow);

    UpdateWindow(editor.Window());



    //головний цикл обробки повідомлень

    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        TranslateMessage(&msg);

        DispatchMessage(&msg);
```

```
    }


    return (int)msg.wParam;

}
```

editor.h:

```
#pragma once

#include <windows.h>

#include <commctrl.h> //бібліотека для Toolbar

#include "shape.h"


class EditorWindow {

private:

    HWND m_hWnd;

    HWND m_hToolBar; //вказівник на панель інструментів

    HINSTANCE m_hInstance;

    int m_currentShape;

    bool m_isDrawing;

    long m_xStart, m_yStart, m_xEnd, m_yEnd;

    Shape** m_pcshape;

    int m_shapeCount;

    const int m_maxShapes = 112;
```

```
    void DrawRubberBand(long x1, long y1, long x2, long y2, int
shapeType);



public:

    EditorWindow(HINSTANCE hInstance);

    ~EditorWindow();



    bool Create(PCWSTR lpWindowName, DWORD dwStyle, int x =
CW_USEDEFAULT, int y = CW_USEDEFAULT, int nWidth = 800, int
nHeight = 600);

    HWND Window() const { return m_hWnd; }



    static LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg,
WPARAM wParam, LPARAM lParam);



    LRESULT HandleMessage(UINT uMsg, WPARAM wParam, LPARAM
lParam);

    void OnCreate();

    void OnSize();

    void OnNotify(LPARAM lParam);

    void OnCommand(WPARAM wParam, LPARAM lParam);

    void OnLButtonDown(LPARAM lParam);

    void OnMouseMove(LPARAM lParam);

    void OnLButtonUp(LPARAM lParam);

    void OnPaint(HDC hdc);

    void OnInitMenuPopup(HMENU hMenu);

};
```

editor.cpp:

```
#include "editor.h"

#include "resource.h"

#include <cmath>



EditorWindow::EditorWindow(HINSTANCE hInstance)

    : m_hWnd(NULL), m_hToolBar(NULL), m_hInstance(hInstance),
m_currentShape(IDM_SHAPE_POINT),

      m_isDrawing(false), m_xStart(0), m_yStart(0), m_xEnd(0),
m_yEnd(0), m_shapeCount(0) {

    m_pcshape = new Shape*[m_maxShapes];

}



EditorWindow::~EditorWindow() {

    for (int i = 0; i < m_shapeCount; i++) {

        delete m_pcshape[i];

        m_pcshape[i] = nullptr;

    }

    delete[] m_pcshape;

}



bool EditorWindow::Create(PCWSTR lpWindowName, DWORD dwStyle, int
x, int y, int nWidth, int nHeight) {

    WNDCLASSEXW wc = { 0 };

    wc.cbSize = sizeof(WNDCLASSEXW);

    wc.lpfnWndProc = EditorWindow::WindowProc;
```

```
    wc.hInstance = m_hInstance;
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszMenuName = MAKEINTRESOURCEW(IDR_MAIN_MENU);
    wc.lpszClassName = L"Lab3EditorClass";

    RegisterClassExW(&wc);

    m_hWnd = CreateWindowW(
        L"Lab3EditorClass", lpWindowName, dwStyle,
        x, y, nWidth, nHeight,
        NULL, NULL, m_hInstance, this
    );

    return (m_hWnd ? true : false);
}

LRESULT CALLBACK EditorWindow::WindowProc(HWND hwnd, UINT uMsg,
WPARAM wParam, LPARAM lParam) {
    EditorWindow* pThis = nullptr;

    if (uMsg == WM_NCCREATE) {
        CREATESTRUCT* pCreate = (CREATESTRUCT*)lParam;
        pThis = (EditorWindow*)pCreate->lpCreateParams;
        SetWindowLongPtr(hwnd, GWLP_USERDATA, (LONG_PTR)pThis);
```

```
        pThis->m_hWnd = hwnd;
    } else {
        pThis = (EditorWindow*)GetWindowLongPtr(hwnd,
GWLP_USERDATA);
    }

    if (pThis) {
        return pThis->HandleMessage(uMsg, wParam, lParam);
    }
    return DefWindowProcW(hwnd, uMsg, wParam, lParam);
}

LRESULT EditorWindow::HandleMessage(UINT uMsg, WPARAM wParam,
LPARAM lParam) {
    switch (uMsg) {
    case WM_CREATE:
        OnCreate();
        break;
    case WM_SIZE:
        OnSize();
        break;
    case WM_NOTIFY:
        OnNotify(lParam);
        break;
    case WM_INITMENUPOPUP:
        OnInitMenuPopup((HMENU)wParam);
```

```
        break;
    case WM_COMMAND:
        OnCommand(wParam, lParam);
        break;
    case WM_LBUTTONDOWN:
        OnLButtonDown(lParam);
        break;
    case WM_MOUSEMOVE:
        OnMouseMove(lParam);
        break;
    case WM_LBUTTONUP:
        OnLButtonUp(lParam);
        break;
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(m_hWnd, &ps);
        OnPaint(hdc);
        EndPaint(m_hWnd, &ps);
        break;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        break;
    default:
        return DefWindowProcW(m_hWnd, uMsg, wParam, lParam);
```

```
    }
    return 0;
}

void EditorWindow::OnCreate() {
    //ініціалізація загальних елементів керування (для Toolbar)
    INITCOMMONCONTROLSEX icex;
    icex.dwSize = sizeof(INITCOMMONCONTROLSEX);
    icex.dwICC  = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icex);

    //створення панелі інструментів із підтримкою підказок
    m_hToolBar = CreateWindowExW(0, TOOLBARCLASSNAMEW, NULL,
        WS_CHILD | WS_VISIBLE | TBSTYLE_TOOLTIPS, 0, 0, 0, 0,
        m_hWnd, (HMENU)2000, m_hInstance, NULL);

    SendMessage(m_hToolBar, TB_BUTTONSTRUCTSIZE,
(WPARAM)sizeof(TBBUTTON), 0);

    //додавання стандартних іконок Windows (заглушки для фігур)
    TBADDBITMAP tbab;
    tbab.hInst = HINST_COMMCTRL;
    tbab.nID = IDB_STD_SMALL_COLOR;
    SendMessage(m_hToolBar, TB_ADDBITMAP, 0, (LPARAM)&tbab);
```

```
    TBBUTTON tbb[4];

    ZeroMemory(tbb, sizeof(tbb));


    tbb[0].iBitmap = STD_FILENEW;

    tbb[0].fsState = TBSTATE_ENABLED;

    tbb[0].fsStyle = TBSTYLE_BUTTON;

    tbb[0].idCommand = IDM_SHAPE_POINT;


    tbb[1].iBitmap = STD_FILEOPEN;

    tbb[1].fsState = TBSTATE_ENABLED;

    tbb[1].fsStyle = TBSTYLE_BUTTON;

    tbb[1].idCommand = IDM_SHAPE_LINE;


    tbb[2].iBitmap = STD_FILESAVE;

    tbb[2].fsState = TBSTATE_ENABLED;

    tbb[2].fsStyle = TBSTYLE_BUTTON;

    tbb[2].idCommand = IDM_SHAPE_RECT;


    tbb[3].iBitmap = STD_PRINTPRE;

    tbb[3].fsState = TBSTATE_ENABLED;

    tbb[3].fsStyle = TBSTYLE_BUTTON;

    tbb[3].idCommand = IDM_SHAPE_ELLIPSE;


    SendMessage(m_hToolBar, TB_ADDBUTTONS, 4, (LPARAM)&tbb);

}
```

```
void EditorWindow::OnSize() {
    SendMessage(m_hToolBar, TB_AUTOSIZE, 0, 0);
}

//обробник OnNotify для спливаючих підказок
void EditorWindow::OnNotify(LPARAM lParam) {
    LPNMHDR lpnm = (LPNMHDR)lParam;
    if (lpnm->code == TTN_GETDISPINFOW) {
        LPTOOLTIPTEXTW lpttt = (LPTOOLTIPTEXTW)lParam;
        lpttt->hinst = m_hInstance;
        switch (lpttt->hdr.idFrom) {
        case IDM_SHAPE_POINT:
            lpttt->lpszText = const_cast<LPWSTR>(L"Крапка");
            break;
        case IDM_SHAPE_LINE:
            lpttt->lpszText = const_cast<LPWSTR>(L"Лінія");
            break;
        case IDM_SHAPE_RECT:
            lpttt->lpszText = const_cast<LPWSTR>(L"Прямокутник");
            break;
        case IDM_SHAPE_ELLIPSE:
            lpttt->lpszText = const_cast<LPWSTR>(L"Еліпс");
            break;
        }
```

```
    }

}



void EditorWindow::OnInitMenuPopup(HMENU hMenu) {

    CheckMenuItem(hMenu, IDM_SHAPE_POINT, MF_BYCOMMAND |
(m_currentShape == IDM_SHAPE_POINT ? MF_CHECKED : MF_UNCHECKED));

    CheckMenuItem(hMenu, IDM_SHAPE_LINE, MF_BYCOMMAND |
(m_currentShape == IDM_SHAPE_LINE ? MF_CHECKED : MF_UNCHECKED));

    CheckMenuItem(hMenu, IDM_SHAPE_RECT, MF_BYCOMMAND |
(m_currentShape == IDM_SHAPE_RECT ? MF_CHECKED : MF_UNCHECKED));

    CheckMenuItem(hMenu, IDM_SHAPE_ELLIPSE, MF_BYCOMMAND |
(m_currentShape == IDM_SHAPE_ELLIPSE ? MF_CHECKED :
MF_UNCHECKED));

}



void EditorWindow::OnCommand(WPARAM wParam, LPARAM lParam) {

    switch (LOWORD(wParam)) {

    case IDM_SHAPE_POINT:

        m_currentShape = IDM_SHAPE_POINT;

        break;

    case IDM_SHAPE_LINE:

        m_currentShape = IDM_SHAPE_LINE;

        break;

    case IDM_SHAPE_RECT:

        m_currentShape = IDM_SHAPE_RECT;

        break;

    case IDM_SHAPE_ELLIPSE:
```

```
        m_currentShape = IDM_SHAPE_ELLIPSE;
        break;
    case IDM_FILE_EXIT:
        DestroyWindow(m_hWnd);
        break;
    }
}

void EditorWindow::DrawRubberBand(long x1, long y1, long x2, long
y2, int shapeType) {
    HDC hdc = GetDC(m_hWnd);
    int oldROP = SetROP2(hdc, R2_NOTXORPEN);
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc,
GetStockObject(NULL_BRUSH));

    if (shapeType == IDM_SHAPE_POINT) {
        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x1 + 1, y1 + 1);
    } else if (shapeType == IDM_SHAPE_LINE) {
        MoveToEx(hdc, x1, y1, NULL);
        LineTo(hdc, x2, y2);
    } else if (shapeType == IDM_SHAPE_RECT) {
        Rectangle(hdc, x1, y1, x2, y2);
    } else if (shapeType == IDM_SHAPE_ELLIPSE) {
```

```
        long dx = std::abs(x2 - x1);
        long dy = std::abs(y2 - y1);
        Ellipse(hdc, x1 - dx, y1 - dy, x1 + dx, y1 + dy);
    }

    SelectObject(hdc, hOldBrush);
    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
    SetROP2(hdc, oldROP);
    ReleaseDC(m_hWnd, hdc);
}

void EditorWindow::OnLButtonDown(LPARAM lParam) {
    if (m_shapeCount < m_maxShapes) {
        m_isDrawing = true;
        m_xStart = LOWORD(lParam);
        m_yStart = HIWORD(lParam);
        m_xEnd = m_xStart;
        m_yEnd = m_yStart;
        SetCapture(m_hWnd);
    }
}

void EditorWindow::OnMouseMove(LPARAM lParam) {
    if (m_isDrawing) {
```

```
        DrawRubberBand(m_xStart, m_yStart, m_xEnd, m_yEnd,
m_currentShape);

        m_xEnd = LOWORD(lParam);

        m_yEnd = HIWORD(lParam);

        DrawRubberBand(m_xStart, m_yStart, m_xEnd, m_yEnd,
m_currentShape);

    }

}



void EditorWindow::OnLButtonUp(LPARAM lParam) {

    if (m_isDrawing) {

        DrawRubberBand(m_xStart, m_yStart, m_xEnd, m_yEnd,
m_currentShape);

        m_xEnd = LOWORD(lParam);

        m_yEnd = HIWORD(lParam);

        m_isDrawing = false;

        ReleaseCapture();



        if (m_shapeCount < m_maxShapes) {

            Shape* pShape = nullptr;

            if (m_currentShape == IDM_SHAPE_POINT) pShape = new
PointShape();

            else if (m_currentShape == IDM_SHAPE_LINE) pShape =
new LineShape();

            else if (m_currentShape == IDM_SHAPE_RECT) pShape =
new RectangleShape();

            else if (m_currentShape == IDM_SHAPE_ELLIPSE) pShape
= new EllipseShape();
```

```
            if (pShape) {
                pShape->SetCoord(m_xStart, m_yStart, m_xEnd,
m_yEnd);
                m_pcshape[m_shapeCount++] = pShape;
                InvalidateRect(m_hWnd, NULL, TRUE);
            }
        }
    }
}

void EditorWindow::OnPaint(HDC hdc) {
    for (int i = 0; i < m_shapeCount; i++) {
        if (m_pcshape[i]) {
            m_pcshape[i]->Show(hdc);
        }
    }
}
```

shape.h:

```
#pragma once
#include <windows.h>

class Shape {
```

```
protected:
    long xs1, ys1, xs2, ys2;
public:
    virtual ~Shape() {}
    void SetCoord(long x1, long y1, long x2, long y2);
    virtual void Show(HDC hdc) = 0;
};
class PointShape : public Shape {
public:
    void Show(HDC hdc) override;
};
class LineShape : public Shape {
public:
    void Show(HDC hdc) override;
};
class RectangleShape : public Shape {
public:
    void Show(HDC hdc) override;
};
class EllipseShape : public Shape {
public:
    void Show(HDC hdc) override;
};
```

shape.cpp:

```
#include "shape.h"

#include <cmath>



void Shape::SetCoord(long x1, long y1, long x2, long y2) {

    xs1 = x1;

    ys1 = y1;

    xs2 = x2;

    ys2 = y2;

}



//крапка

void PointShape::Show(HDC hdc) {

    SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));

}

//лінія

void LineShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);



    MoveToEx(hdc, xs1, ys1, NULL);

    LineTo(hdc, xs2, ys2);



    SelectObject(hdc, hOldPen);

    DeleteObject(hPen);
```

```
}

//прямокутник(12 вар:ввід по 2 протилежних кутах,жовте
заповнення)

void RectangleShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 0)); //жовтий
колір

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);



    Rectangle(hdc, xs1, ys1, xs2, ys2);



    SelectObject(hdc, hOldPen);

    SelectObject(hdc, hOldBrush);

    DeleteObject(hPen);

    DeleteObject(hBrush);

}

//еліпс(12 вар:ввід від центру до кута,без заповнення)

void EllipseShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = (HBRUSH)GetStockObject(NULL_BRUSH);// прозоре
заповнення

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);



    long dx = std::abs(xs2 - xs1);
```

```
    long dy = std::abs(ys2 - ys1);

    Ellipse(hdc, xs1 - dx, ys1 - dy, xs1 + dx, ys1 + dy);



    SelectObject(hdc, hOldPen);

    SelectObject(hdc, hOldBrush);

    DeleteObject(hPen);

}
```

resource.h:

```
#pragma once



#define IDR_MAIN_MENU                   101

#define IDM_FILE_EXIT                   40001

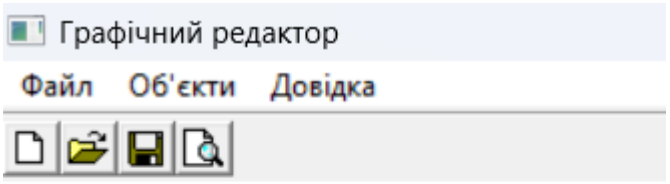
#define IDM_SHAPE_POINT                 40002

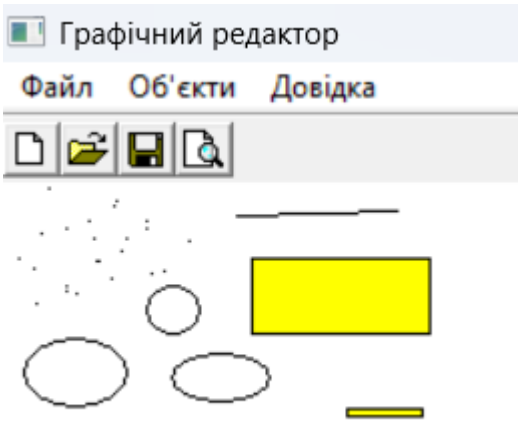
#define IDM_SHAPE_LINE                  40003

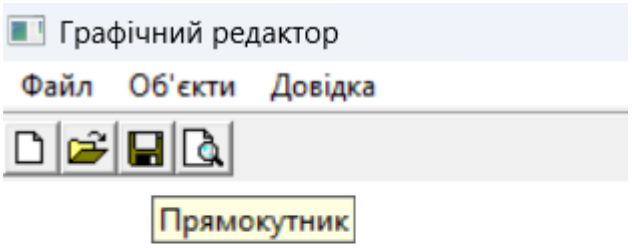
#define IDM_SHAPE_RECT                  40004

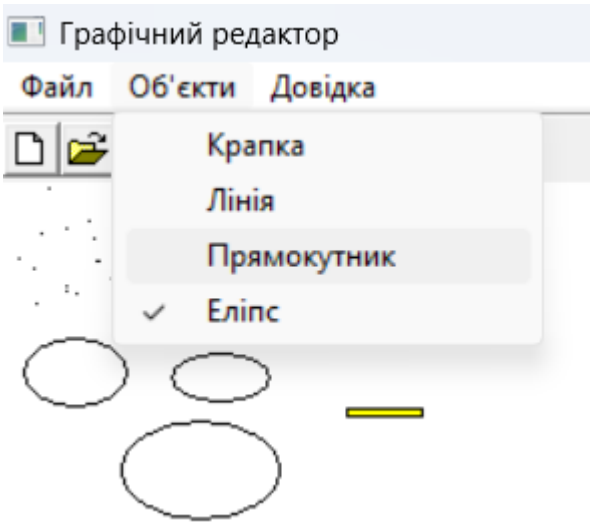
#define IDM_SHAPE_ELLIPSE               40005
```

Скріншоти виконаної роботи:

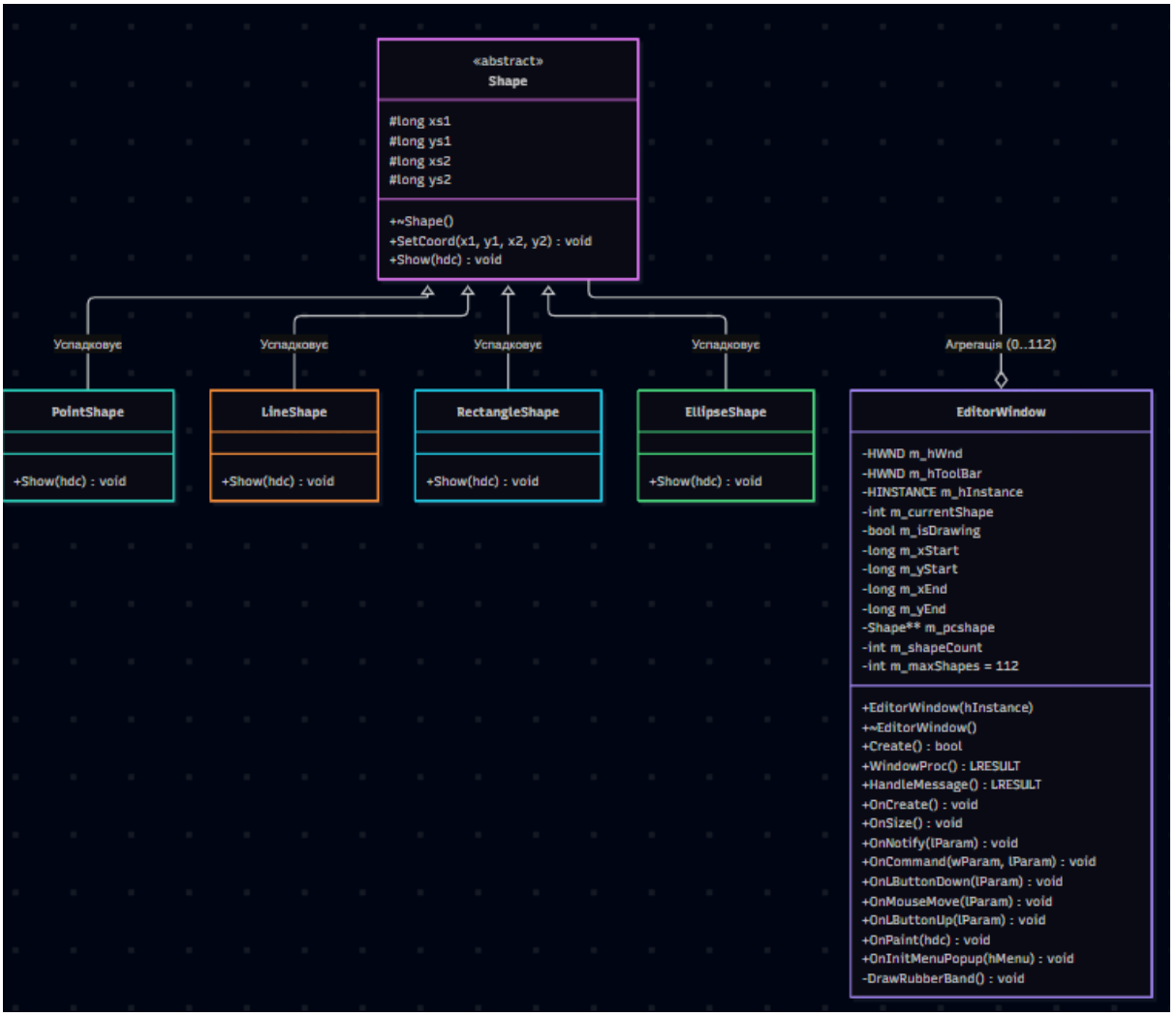








Діаграма класів:


«abstract»
Shape
#long xs1
#long ys1
#long xs2
#long ys2
+~Shape()
+SetCoord(x1, y1, x2, y2) : void
+Show(hdc) : void
Успадковує
Успадковує
Успадковує
Успадковує
Агрегація (0..112)
PointShape
+Show(hdc) : void
LineShape
+Show(hdc) : void
RectangleShape
+Show(hdc) : void
EllipseShape
+Show(hdc) : void
EditorWindow
-HWND m_hWnd
-HWND m_hToolBar
-HINSTANCE m_hInstance
-int m_currentShape
-bool m_isDrawing
-long m_xStart
-long m_yStart
-long m_xEnd
-long m_yEnd
-Shape** m_pcshape
-int m_shapeCount
-int m_maxShapes = 112
+EditorWindow(hInstance)
+~EditorWindow()
+Create() : bool
+WindowProc() : LRESULT
+HandleMessage() : LRESULT
+OnCreate() : void
+OnSize() : void
+OnNotify(lParam) : void
+OnCommand(wParam, lParam) : void
+OnLButtonDown(lParam) : void
+OnMouseMove(lParam) : void
+OnLButtonUp(lParam) : void
+OnPaint(hdc) : void
+OnInitMenuPopup(hMenu) : void
-DrawRubberBand() : void

Висновок: в ході виконання лабораторної роботи №3 я отримав вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.